SQL BASICS

Corrected Study Guide for RAG Retrieval

Contents

1. 1. Relational Database and Tables
2. 2. SELECT and FROM
3. 3. WHERE and Operators
4. 4. NULL and Three-Valued Logic
5. 5. ORDER BY and LIMIT
6. 6. Aggregate Functions
7. 7. GROUP BY and HAVING
8. 8. CASE Expressions
9. 9. INSERT, UPDATE, and DELETE
10. 10. Constraints and Keys
11. 11. Quick Revision and Practice

Purpose: a structured study guide designed for topic-wise learning and retrieval practice.

1. Relational Database and Tables

A relational database stores data in tables. A row represents a record and a column represents an attribute. A primary key uniquely identifies each row; a foreign key references a key in another table.

```
CREATE DATABASE practice_db;
USE practice_db;

CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(80) NOT NULL,
    dept_id INT,
    salary DECIMAL(10,2),
    hire_date DATE,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);
```

2. SELECT and FROM

SELECT chooses the columns or expressions to return. FROM identifies the source table. Use explicit column names in application queries rather than SELECT * when a stable output schema matters.

```
SELECT emp_id, emp_name, salary
FROM Employees;
```

Use DISTINCT to remove duplicate result rows, not as a substitute for understanding why duplicates exist.

3. WHERE and Operators

```
SELECT emp_name, salary
FROM Employees
WHERE salary >= 50000
  AND dept_id IN (10, 20);
```

Common operators include =, <>, <, <=, >, >=, BETWEEN, IN, LIKE, AND, OR, and NOT. BETWEEN is inclusive at both ends in standard SQL behavior. Use parentheses when mixing AND and OR.

4. NULL and Three-Valued Logic

NULL represents missing, unknown, or inapplicable data. Comparisons such as salary = NULL do not work as intended. Use IS NULL or IS NOT NULL.

```
SELECT emp_name
FROM Employees
WHERE dept_id IS NULL;
```

A condition involving NULL may evaluate to UNKNOWN rather than TRUE or FALSE. WHERE keeps only rows whose condition is TRUE.

5. ORDER BY and LIMIT

ORDER BY sorts the result. ASC is ascending and DESC is descending. LIMIT restricts the number of returned rows in MySQL.

```
SELECT emp_name, salary
FROM Employees
ORDER BY salary DESC, emp_name ASC
LIMIT 5;
```

Always use a deterministic tie-breaker when the exact order matters. LIMIT without ORDER BY does not guarantee which rows are selected.

6. Aggregate Functions

Aggregate functions summarize multiple rows. Common functions are COUNT, SUM, AVG, MIN, and MAX. Most aggregates ignore NULL values; COUNT(*) counts rows, while COUNT(column) counts non-NULL values in that column.

```
SELECT COUNT(*) AS employee_count,
       AVG(salary) AS average_salary,
       MIN(salary) AS lowest_salary,
       MAX(salary) AS highest_salary
FROM Employees;
```

7. GROUP BY and HAVING

GROUP BY creates groups for aggregation. HAVING filters groups after aggregate values are computed; WHERE filters individual rows before grouping.

```
SELECT dept_id, COUNT(*) AS employee_count,
       AVG(salary) AS average_salary
FROM Employees
WHERE salary IS NOT NULL
GROUP BY dept_id
HAVING COUNT(*) >= 2
ORDER BY average_salary DESC;
```

In portable SQL, every selected non-aggregated expression should be validly represented in the GROUP BY or be functionally dependent on grouped keys under the database's rules.

8. CASE Expressions

CASE creates conditional values inside a query. It is useful for labels, buckets, and conditional aggregation.

```
SELECT emp_name, salary,
       CASE
           WHEN salary >= 80000 THEN 'High'
           WHEN salary >= 50000 THEN 'Medium'
           WHEN salary IS NOT NULL THEN 'Low'
           ELSE 'Unknown'
       END AS salary_band
FROM Employees;
```

9. INSERT, UPDATE, and DELETE

```
INSERT INTO Departments (dept_id, dept_name)
VALUES (10, 'Engineering');

UPDATE Employees
SET salary = salary * 1.05
WHERE dept_id = 10;

DELETE FROM Employees
WHERE emp_id = 101;
```

Always inspect the WHERE clause before UPDATE or DELETE. Without it, every row may be changed or removed. Use transactions when a multi-step operation must succeed or fail as a unit.

10. Constraints and Keys

• PRIMARY KEY: unique and not NULL row identifier.

• FOREIGN KEY: enforces a reference to a valid key, subject to the database's constraint rules.

• NOT NULL: rejects missing values.

• UNIQUE: prevents duplicate values or combinations, subject to NULL semantics in the DBMS.

• CHECK: restricts values using a Boolean condition where supported.

• DEFAULT: supplies a value when an insert omits the column.

11. Quick Revision and Practice

Query order concept

A useful logical processing model is FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, then LIMIT. This is a conceptual model, not a promise about physical execution order.

Practice questions

• Find employees earning at least 50,000, highest salary first.

• Count employees in each department and show only departments with at least five employees.

• Find employees with no department assigned.

• Return the three highest salaries with employee names.

• Explain COUNT(*) versus COUNT(salary).

• Classify salaries into High, Medium, and Low using CASE.

• Why is WHERE COUNT(*) >= 2 invalid in a typical grouped query, and where should the condition go?